

UNIVERSIDAD TÉCNICA PARTICULAR DE LOJA

Departamento de Ciencias de la Computación
Carrera de Ingeniería en Ciencias de la Computación

PROYECTO BIMESTRAL

Análisis del Reto: Plataforma de Marketplace Mayorista B2B/B2C

Autores:

David León

Víctor Mendoza

Docente:

Ing. René Rolando Elizalde Solano

Asignatura:

Plataformas Web

Periodo y Paralelo:

Abril - Agosto 2026, Paralelo A

Loja - Ecuador

Junio 2026

Índice general

1. Análisis del Reto	2
1.1. Introducción	2
1.2. Problemática	2
1.3. Metodología o estrategia de desarrollo	3
2. Planificación y análisis	4
2.1. Análisis	4
2.2. Requisitos funcionales	4
2.3. Requisitos no funcionales	5
2.4. Historias de Usuario / Casos de Uso	5
2.5. Sprint	7
2.6. Diagramas: Casos de Uso del Sistema - Preguntas y respuesta al diagrama	8
2.7. Diagrama de componentes	10
2.8. Arquitectura lógica y física (propuesta)	10
2.9. Prototipo de pantallas no funcional	11
2.9.1. Flujo de la Tienda Online (Vista del Tendero / Cliente)	11
2.9.2. Flujo del Vendedor Freelance (Dashboard de Ventas)	12
2.9.3. Flujo de la Empresa Proveedora (Panel de Control de Distribución)	12

Capítulo 1

Análisis del Reto

1.1. Introducción

El sector de la distribución mayorista en el ámbito de negocios Business-to-Business (B2B) y Business-to-Consumer (B2C) se enfrenta a un proceso acelerado de transformación digital. Tradicionalmente, la intermediación comercial se ha basado en dinámicas presenciales o canales analógicos (llamadas telefónicas, hojas de pedido físicas), los cuales son propensos a ineficiencias de stock, errores humanos en facturación y demoras en los cobros.

Para resolver esta brecha, se proyecta el desarrollo de una **Plataforma de Marketplace Mayorista B2B/B2C**. Este sistema digital unifica la interacción comercial entre tres actores principales:

- **Empresas (Proveedores):** Entidades que ofertan mercancía al por mayor (lotes, pacas).
- **Vendedores Freelance:** Agentes comerciales independientes que representan a una o varias empresas para levantar pedidos en el territorio.
- **Tenderos o Clientes Finales:** Comercios minoristas locales que adquieren productos a través de la plataforma de forma autónoma o asistida.

El software propuesto facilita no solo la automatización del ciclo de venta y facturación, sino que gestiona dinámicamente un esquema de comisiones retenidas e incluye evaluaciones técnicas para habilitar vendedores certificados en nichos especializados (como la distribución farmacéutica).

1.2. Problemática

La gestión convencional de pedidos y logística comercial B2B adolece de problemas críticos que restringen el crecimiento de las empresas proveedoras y causan pérdidas a los comerciantes minoristas. Las principales problemáticas identificadas son:

1. **Ventas Dobles y Desfase de Inventario:** La falta de sincronización del stock en tiempo real permite que dos vendedores consoliden pedidos sobre un lote único agotado, provocando anulaciones y descontento en los clientes.

2. **Riesgo en la Cadena de Cobros:** Los pedidos mayoristas representan montos elevados. La ausencia de un canal seguro de pagos por adelantado (esquemas de 50 % o 100 %) expone a las empresas a impagos o cancelaciones tardías de mercancía ya despachada.
3. **Informalidad en la Red de Ventas Freelance:** Al no existir un control automatizado de comisiones y entregas, el cálculo manual de la retribución de los vendedores freelance se vuelve complejo y opaco. Asimismo, las empresas corren riesgos al delegar la comercialización de productos regulados a personal sin la debida capacitación.
4. **Brecha Tecnológica (Analfabetismo Digital):** El perfil demográfico del comerciante local (tendero) suele presentar resistencia al uso de plataformas complejas. Diseños con interfaces sobrecargadas o flujos extensos de compra incrementan la tasa de abandono de los pedidos.

1.3. Metodología o estrategia de desarrollo

Para hacer frente a estas problemáticas y construir un producto robusto con entregas incrementales, se ha seleccionado el marco de trabajo ágil **Scrum** [7]. Esto permite iterar ágilmente sobre las especificaciones de requisitos funcionales y realizar demostraciones continuas al usuario final.

La estrategia tecnológica se basa en decisiones arquitectónicas enfocadas en la transaccionalidad, la escalabilidad y la personalización estética [6]:

- **Core de Desarrollo:** Arquitectura desacoplada basada en **Next.js** (React) con TypeScript para la interfaz frontend y **Django** (Python) con **Django REST Framework (DRF)** en el backend. Esta separación divide de forma limpia el desarrollo del cliente interactivo y las API de lógica de negocio del servidor.
- **Capa de Persistencia:** **PostgreSQL** como base de datos relacional. Su motor garantiza soporte nativo de transacciones ACID (Atomicity, Consistency, Isolation, Durability), indispensable para realizar bloqueos de fila (*row locks*) que previenen la doble reserva de inventario de manera concurrente.
- **Acceso a Datos:** **Django ORM** para gestionar la comunicación con la base de datos PostgreSQL, facilitando la creación de modelos, migraciones de base de datos seguras y un panel administrativo integrado listo para producción.
- **Diseño de Interfaz:** **Vanilla CSS** empleando un sistema de variables y tokens globales. Se evita el uso de frameworks utilitarios como Tailwind CSS para mantener un control absoluto sobre el peso del bundle y garantizar una experiencia de usuario (UX) sumamente fluida con microanimaciones e interactividad adaptada a pantallas táctiles de dispositivos móviles.

Capítulo 2

Planificación y análisis

2.1. Análisis

La plataforma web del Marketplace funciona bajo un modelo operativo de tres niveles comerciales con facturación integrada. Las empresas proveedoras son las propietarias del catálogo mayorista y definen un monto mínimo de compra (entre \$60 y \$80) para hacer rentable el envío logístico. El sistema monetiza a través de un esquema doble: cobro de suscripción anual a los proveedores o aplicación de un porcentaje fijo (2 %) sobre el valor total de cada transacción finalizada.

El flujo de comisión para los vendedores independientes está integrado al ciclo de entrega. Cuando el vendedor registra una transacción, la comisión se calcula de acuerdo con los márgenes parametrizados por el proveedor. Dicha comisión permanece en estado Retenida.^{en} la base de datos hasta que el tendero confirma explícitamente haber recibido la mercancía, momento en el cual se libera la comisión para su respectiva transferencia, garantizando así la calidad del servicio.

2.2. Requisitos funcionales

A continuación, se tabulan los requisitos funcionales (RF) extraídos de la especificación técnica del proyecto:

Código	Descripción del Requisito Funcional
RF1.1	Registro e inicio de sesión seguro con roles diferenciados: Administrador, Empresa (Proveedor), Vendedor Freelance y Tendero (Cliente).
RF1.2	Configuración empresarial del perfil de vendedor freelance requerido (Perfil General o Perfil Calificado).
RF1.3	Módulo de exámenes y certificaciones en línea para habilitar a vendedores freelance en perfiles calificados.
RF2.1	Visualización dinámica de catálogo mayorista orientado a paquetes cerrados, lotes y pacas.
RF2.2	Control y visualización del stock en tiempo real por cada almacén de proveedor.
RF2.3	Alertas y notificaciones automáticas ante niveles de stock inferiores al umbral de seguridad.
RF2.4	Sincronización del inventario mediante APIs o webhooks con ERPs o sistemas contables externos de los proveedores.
RF3.1	Compra directa autogestionada por Tenderos que cumplan con el monto mínimo.
RF3.2	Levantamiento de pedidos presencial por Vendedores Freelance a nombre de Tenderos.
RF3.3	Configuración dinámica por parte de las empresas de montos mínimos de compra.
RF3.4	Generación y envío automático de facturación detallada tras cada orden.
RF4.1	Pasarela de pagos integrada para abono de anticipo (50 %) o pago total (100 %).
RF4.2	Cálculo automatizado del porcentaje de comisión del freelance según márgenes establecidos.
RF4.3	Retención y liberación condicional de comisiones sujeta a la entrega efectiva del pedido.
RF5.1	Cobro automatizado de comisión transaccional (2 %) o membresía anual a los proveedores.

Tabla 2.1: Matriz de Requisitos Funcionales del Sistema.

2.3. Requisitos no funcionales

Los requisitos no funcionales (RNF) definen las restricciones de rendimiento, usabilidad, seguridad y plataforma tecnológica del sistema:

2.4. Historias de Usuario / Casos de Uso

A continuación se formulan historias de usuario prioritarias que guían el desarrollo de la plataforma:

Historia de Usuario 1: Registro de Pedido por Freelance

Como Vendedor Freelance,
Quiero registrar un pedido a nombre de un Tendero local desde mi interfaz móvil,
Para agilizar su abastecimiento e incrementar mi registro de comisiones.
Criterios de Aceptación:

Código	Descripción del Requisito No Funcional
RNF1.1	Restricción de Canal: El sistema se desplegará y optimizará inicialmente en formato de aplicación web interactiva (no nativa).
RNF1.2	Integrabilidad: Diseño modular basado en microservicios o API REST para facilitar enlaces contables fluidos.
RNF2.1	Usabilidad Simplificada: La UI para los Tenderos debe contar con tipografía de alta legibilidad, botones grandes de mínimo 48px y un flujo de compra rápido enfocado en reducir la fricción digital.
RNF2.2	Flujo de Menor Clics: El proceso desde la selección de artículos hasta el pago final debe completarse en un máximo de 3 pasos guiados.
RNF3.1	Seguridad Transaccional: Cumplimiento estricto de la norma PCI-DSS para transacciones mediante cifrado en Stripe.
RNF3.2	Protección de Datos: Cifrado irreversible (hashing con bcrypt) de credenciales del usuario en la persistencia.
RNF4.1	Tiempo Real: Actualización de inventario con latencia menor a 1.5 segundos ante transacciones concurrentes.
RNF4.2	Escalabilidad: El esquema relacional de PostgreSQL y los modelos de Django ORM deben diseñarse bajo estándares que permitan una transición directa a APIs móviles.

Tabla 2.2: Matriz de Requisitos No Funcionales del Sistema.

- El freelance debe buscar y seleccionar al Tendero cliente mediante su código de local o nombre.
- La interfaz debe validar que el pedido supere el monto mínimo de compra del proveedor seleccionado.
- El pedido se crea con estado "Pendiente de Confirmación de Entrega".

Historia de Usuario 2: Compra Directa Adaptada para Tendero

Como Tendero con bajo conocimiento digital,
Quiero realizar una compra directa desde un catálogo simple con botones amplios,
Para surtir mi negocio sin depender de una llamada telefónica o visita física.
Criterios de Aceptación:

- Visualizar fotos de productos agrupadas por categorías claras.
- Agregar artículos con botones flotantes de "+" y "-" de gran escala.
- Visualizar el resumen final con el cobro del 50 % inicial de manera simplificada en una sola pantalla.

Historia de Usuario 3: Creación de Test de Certificación

Como Administrador de Empresa Proveedora de Productos Calificados (ej. Farmacéuticos),
Quiero diseñar un examen interactivo de certificación dentro de la plataforma,

Para garantizar que solo los vendedores freelance aprobados distribuyan mi stock de medicamentos.

Criterios de Aceptación:

- Formulario para ingresar preguntas de opción múltiple con puntaje asignado.
- Establecer la nota mínima de aprobación (ej. 80 %).
- Los freelancers reprobados son bloqueados temporalmente para aplicar a la distribución de esa empresa.

2.5. Sprint

La planificación técnica de implementación se segmenta en dos Sprints iniciales de dos semanas cada uno:

1. Sprint 1: Inicialización y Core Arquitectónico

- Definición de la estructura de carpetas en el cliente Next.js y en el backend Django.
- Diseño e implementación de modelos Django y migraciones en PostgreSQL.
- Configuración de la autenticación mediante tokens en Django REST Framework.
- Creación de comandos de semilla (fixtures/seeder) en Django para usuarios y perfiles iniciales.
- Creación del Layout CSS global con variables de diseño premium.

2. Sprint 2: Catálogo Mayorista y Sistema de Pedidos

- Implementación de vistas de catálogos mayoristas adaptables en Next.js.
- Integración de la API de Django REST Framework con el cliente Next.js.
- Integración de cobros de Stripe en las vistas del backend Django.
- Lógica de retención de comisiones y panel de verificación de entregas en Django.
- Pruebas funcionales concurrentes para validar la integridad en la reducción de stock.

2.6. Diagramas: Casos de Uso del Sistema - Preguntas y respuesta al diagrama

El comportamiento y los requisitos funcionales del sistema se modelan mediante un diagrama de casos de uso, el cual detalla la interacción de los actores (tanto usuarios de la plataforma como sistemas externos) con las funcionalidades principales de la plataforma. A continuación se presenta el diagrama correspondiente:

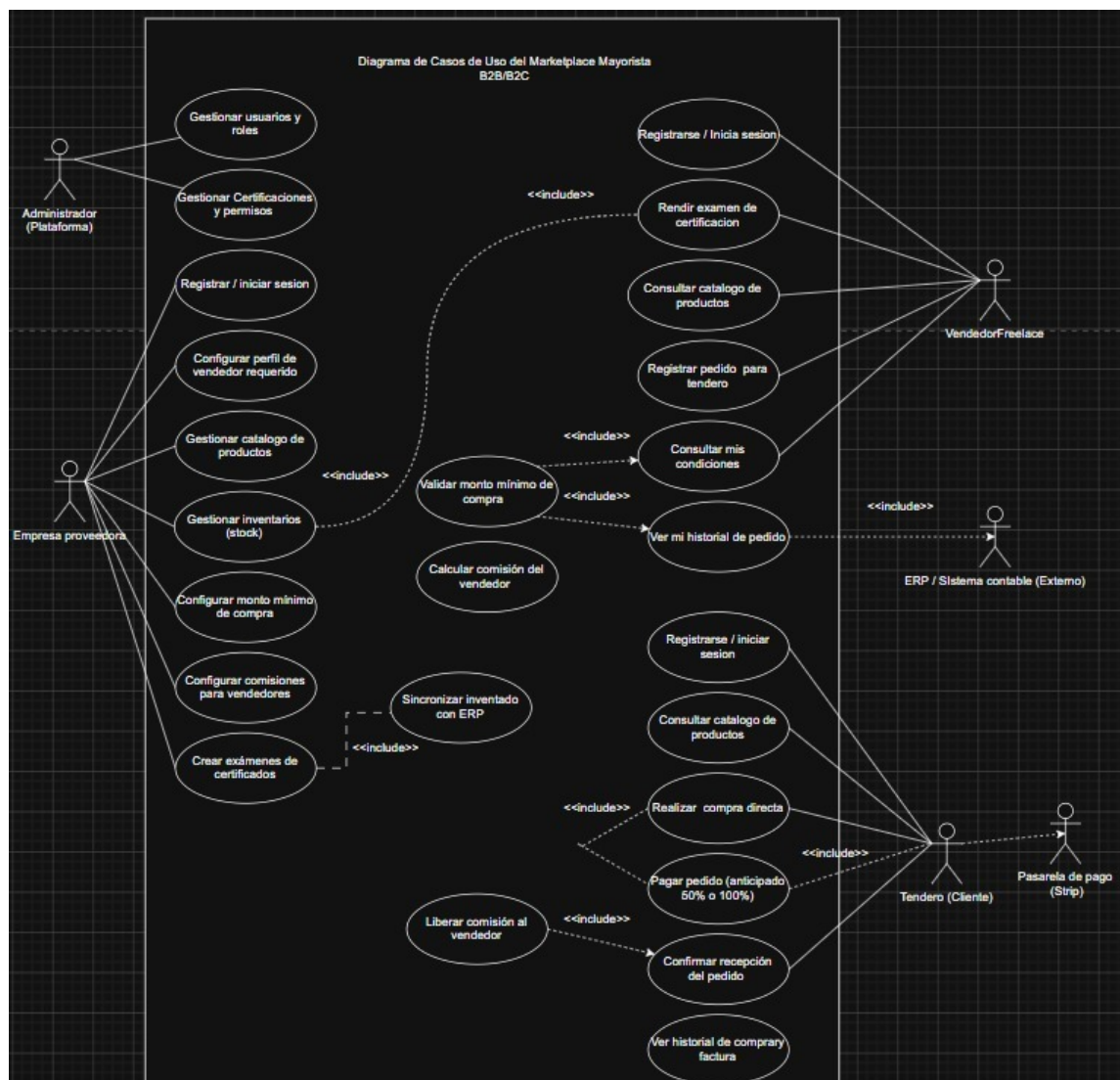


Figura 2.1: Diagrama de Casos de Uso del Marketplace Mayorista B2B/B2C.

Preguntas y Respuestas al Diseño del Diagrama

1. **¿Cómo se gestiona el inventario de forma segura ante pedidos concurrentes?** **Respuesta:** Se utiliza PostgreSQL bajo aislamiento de transacciones o bloqueos pesimistas selectivos mediante el uso de `select_for_update()` en el ORM de Django dentro de una transacción atómica (`@transaction.atomic`). Al iniciar la transacción de compra, Django ORM bloquea la fila del producto en evaluación,

verificando que el stock sea suficiente antes de proceder con el débito y la creación del pedido. Si el stock no cumple, la transacción realiza un rollback completo.

2. **¿De qué manera interactúan las cuentas de usuarios con perfiles de especialización? Respuesta:** La tabla o modelo `User` centraliza la autenticación en Django (extendiendo la clase `AbstractUser`). Dependiendo del valor asignado al campo `role` (`EMPRESA`, `VENDEDOR_FREELANCE`, `TENDERO`, `ADMIN`), el backend enlaza la cuenta a su perfil correspondiente en `CompanyProfile` o `FreelanceProfile` a través de relaciones de tipo `OneToOneField`.
3. **¿Cómo se restringe la venta de un producto que requiere perfil calificado? Respuesta:** Cuando un freelance intenta levantar un pedido de un proveedor con certificaciones activas, el servidor de Django ejecuta un query que verifica la existencia de un registro de aprobación con `passed = True` en el modelo `CertificationAttempt` para el `CertificationTest` correspondiente a dicha empresa. Si no se cumple la condición, el backend retorna una excepción de tipo `PermissionDenied`.
4. **¿Qué rol cumple el actor externo "Pasarela de pagos (Stripe)" cómo se relaciona con el caso de uso "Pagar pedido"? Respuesta:** El actor "Pasarela de pagos (Stripe)"^{es} un sistema externo que interactúa directamente con el caso de uso "Pagar pedido (anticipado 50 % o 100 %)". Tras recibir la orden de pago desde el frontend en Next.js, Django gestiona la sesión de Stripe y, una vez que el usuario ingresa sus datos financieros en el portal seguro de Stripe, esta pasarela notifica el éxito del cobro al backend para validar y procesar la orden.
5. **¿Cómo interactúa el caso de uso "Sincronizar inventario con ERP" con el sistema de la empresa proveedora? Respuesta:** Este caso de uso permite que los inventarios físicos de la empresa proveedora se sincronicen bidireccionalmente. Cuando la empresa realiza cambios en su stock local a través de su ERP externo, este se comunica con el backend del Marketplace mediante la API REST expuesta en Django, garantizando que el catálogo de productos muestre existencias reales de forma automática.

2.7. Diagrama de componentes

El diagrama de componentes detalla la arquitectura modular del software y la comunicación orientada al cliente-servidor:

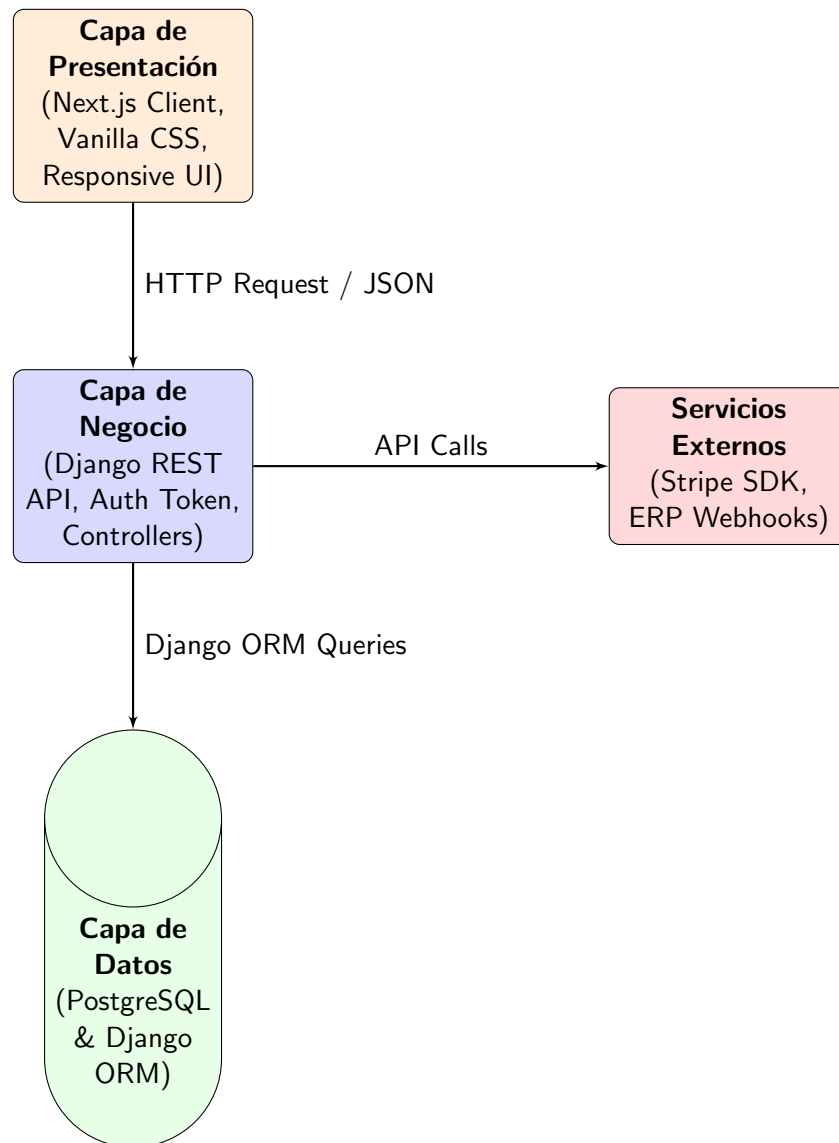


Figura 2.2: Diagrama de Componentes de la Aplicación.

2.8. Arquitectura lógica y física (propuesta)

La propuesta arquitectónica garantiza una alta disponibilidad física y un acoplamiento débil a nivel lógico:

Arquitectura Lógica

Estructurada bajo un patrón de arquitectura desacoplada frontend/backend:

1. **Capa de Interfaz de Usuario (Frontend):** Implementada en Next.js (React), encargada de renderizar las vistas dinámicas de forma responsiva y consumir los endpoints de la API REST backend mediante peticiones asíncronas.
2. **Capa de API y Backend (Lógica de Negocio):** Construida en Django utilizando Django REST Framework (DRF). Maneja la validación de montos mínimos, cálculo de comisiones, autenticación por tokens (JWT o SimpleJWT), control de acceso multi-rol y la comunicación con servicios externos.
3. **Capa de Persistencia:** Gestionada por el ORM de Django, que interactúa directamente con PostgreSQL. Ofrece seguridad integrada ante inyecciones de código y asegura la integridad transaccional mediante decoradores y transacciones atómicas nativas.

Arquitectura Física

El esquema físico propuesto aprovecha entornos especializados para frontend y backend:

- **Hospedaje Frontend:** Vercel o Netlify para alojar y servir la aplicación Next.js de forma estática y optimizada.
- **Hospedaje Backend:** Servidores basados en contenedores o PaaS como **Render**, Heroku o AWS Elastic Beanstalk para ejecutar la aplicación de Django bajo un servidor WSGI/ASGI (como Gunicorn/Uvicorn).
- **Base de Datos Hospedada:** Instancia de PostgreSQL gestionada en la nube mediante **Neon.tech** o Supabase.
- **Pasarelas y Servicios:** Servidores de Stripe para los pagos electrónicos.

2.9. Prototipo de pantallas no funcional

El diseño de la interfaz gráfica prioriza la sencillez, la legibilidad y la experiencia visual premium (con un esquema de colores oscuro, tipografía legible y microinteracciones claras) para reducir la fricción digital en los minoristas y ofrecer herramientas eficientes a las empresas y vendedores. A continuación, se detallan los prototipos de pantalla desarrollados para cada flujo operativo:

2.9.1. Flujo de la Tienda Online (Vista del Tendero / Cliente)

Representa la experiencia de compra directa adaptada para el tendero local, facilitando un proceso sin fricciones y guiado en todo momento.

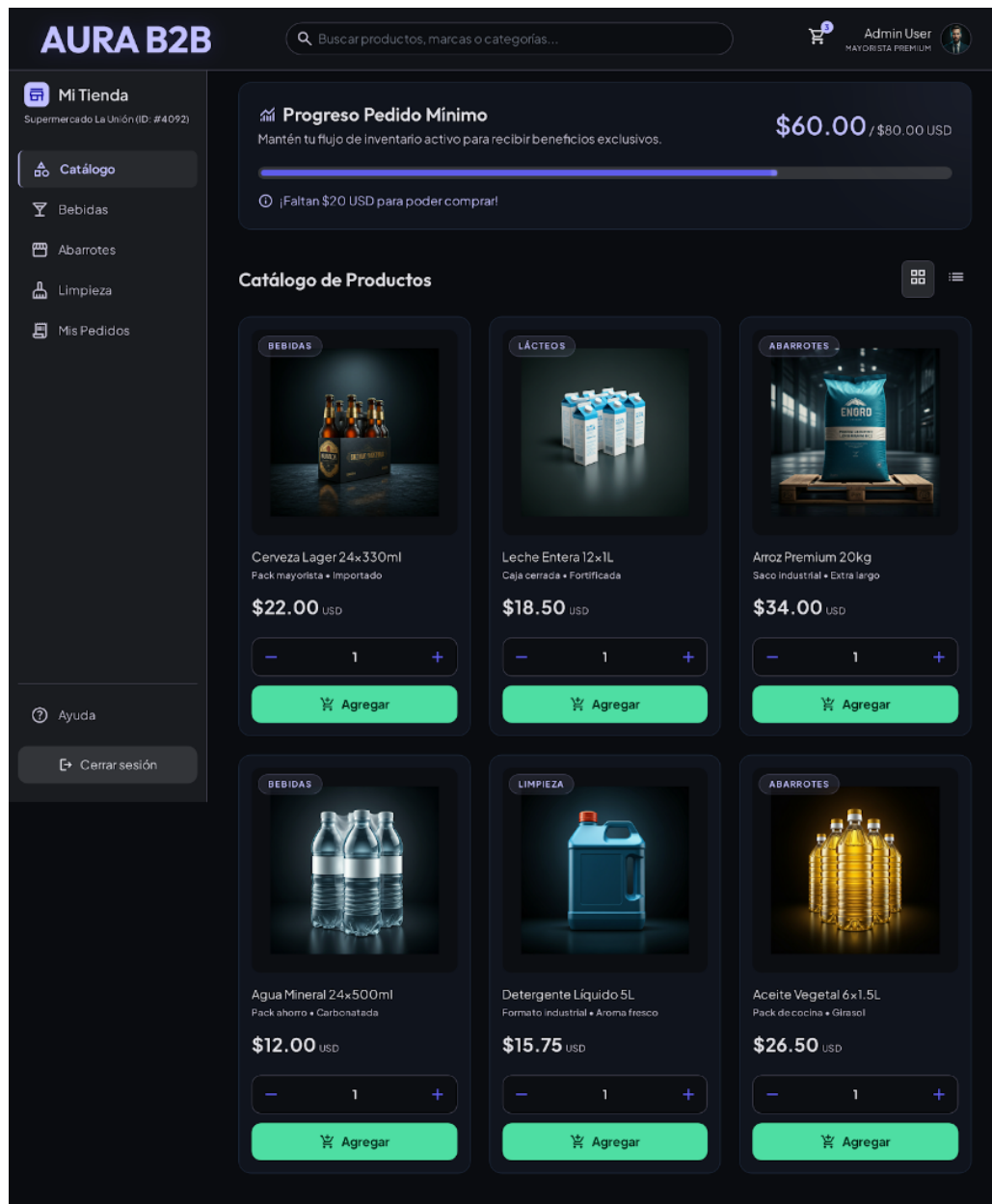


Figura 2.3: Prototipo 1: Catálogo de Productos y barra de progreso del monto mínimo de compra.

2.9.2. Flujo del Vendedor Freelance (Dashboard de Ventas)

Consola de administración donde el vendedor freelance registra sus pedidos a nombre de tenderos, valida sus comisiones y visualiza el estado de sus certificaciones activas.

2.9.3. Flujo de la Empresa Proveedora (Panel de Control de Distribución)

Panel administrativo para la empresa donde se configura el monto mínimo de compra, se confirman pedidos y se monitorean las alertas automáticas de stock.

Finalizar Compra

Aura B2B

Plan de Pago

Pagar 50% por Adelanto

Pagos \$40.00 USD ahora y \$40.00 USD al recibir.

Recomendado

Pagar 100% Completo

Pagos \$80.00 USD ahora.

Información de Pago

NOMBRE EN LA TARJETA

Ej. Juan Pérez

NÚMERO DE TARJETA

0000 0000 0000 0000

FECHA DE EXPIRACIÓN

MM/YY

CVV

123

Confirmar Adelanto (\$40.00)

Tus datos están encriptados y protegidos por protocolos de seguridad bancaria.

Resumen del Pedido

Caja Aceite Cocina

Cantidad: 2 unidades

\$50.00

Arroz Premium (5kg)

Cantidad: 5 unidades

\$30.00

Subtotal

\$80.00

Impuestos (IVA)

\$0.00

Total

\$80.00 USD

Pagos Seguros

Garantía Aura

SSL Encrypt

Aura B2B

© 2024 Aura B2B. Pagos seguros y protegidos.

Términos y Condiciones

Privacidad

Ayuda

Figura 2.4: Prototipo 2: Pantalla de finalización de compra (Checkout) con opciones de pago del 50 % o 100 %.

13

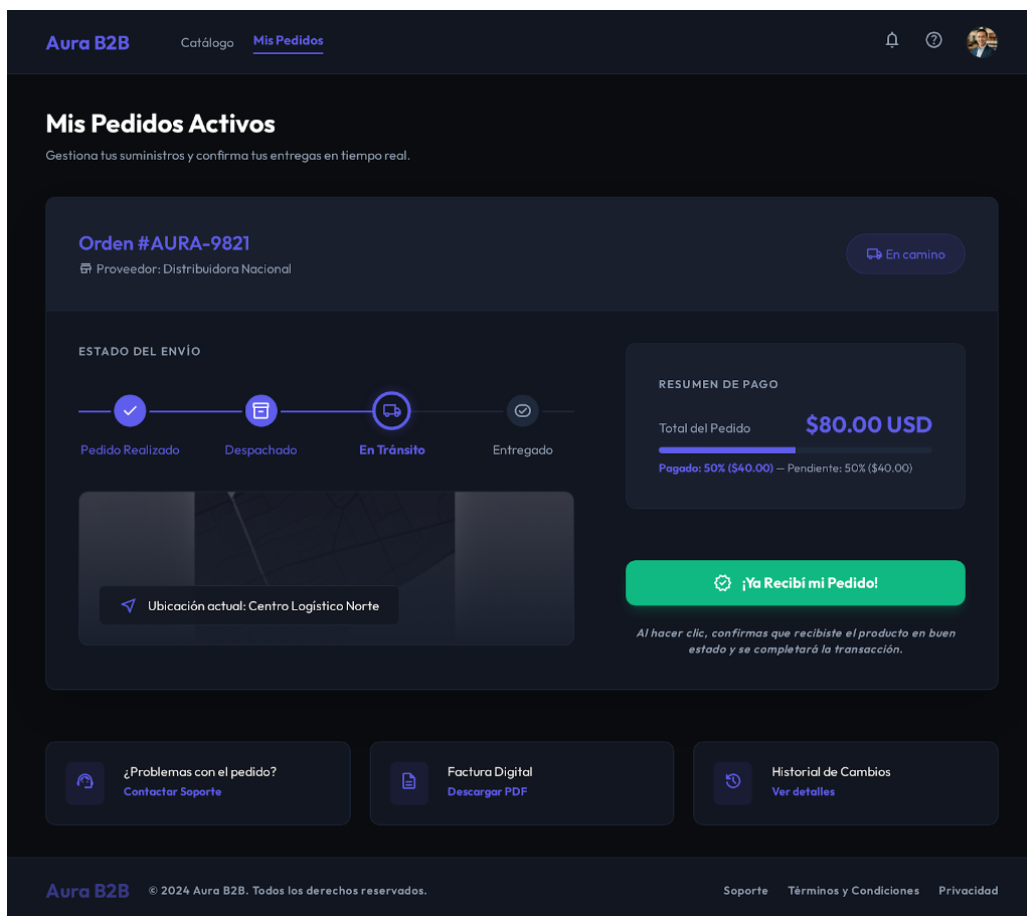


Figura 2.5: Prototipo 3: Rastreo activo de pedidos y confirmación de recepción en tiempo real.

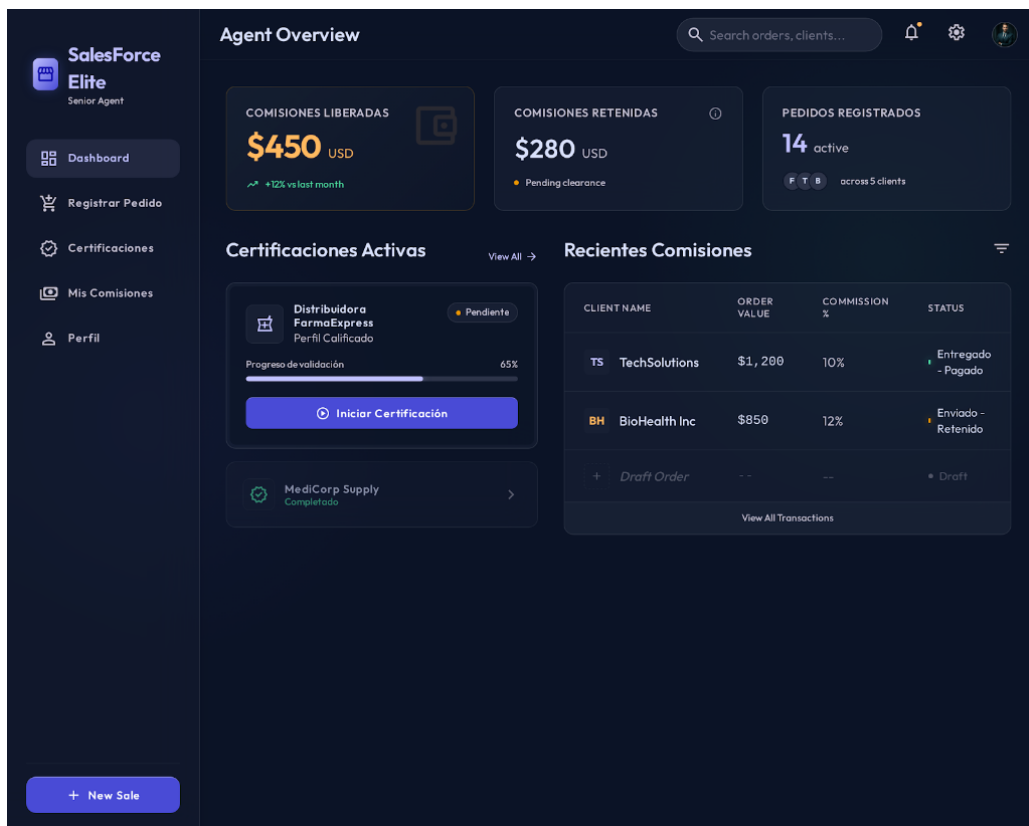


Figura 2.6: Prototipo 4: Dashboard general del Agente Freelance con comisiones y estado de certificaciones.

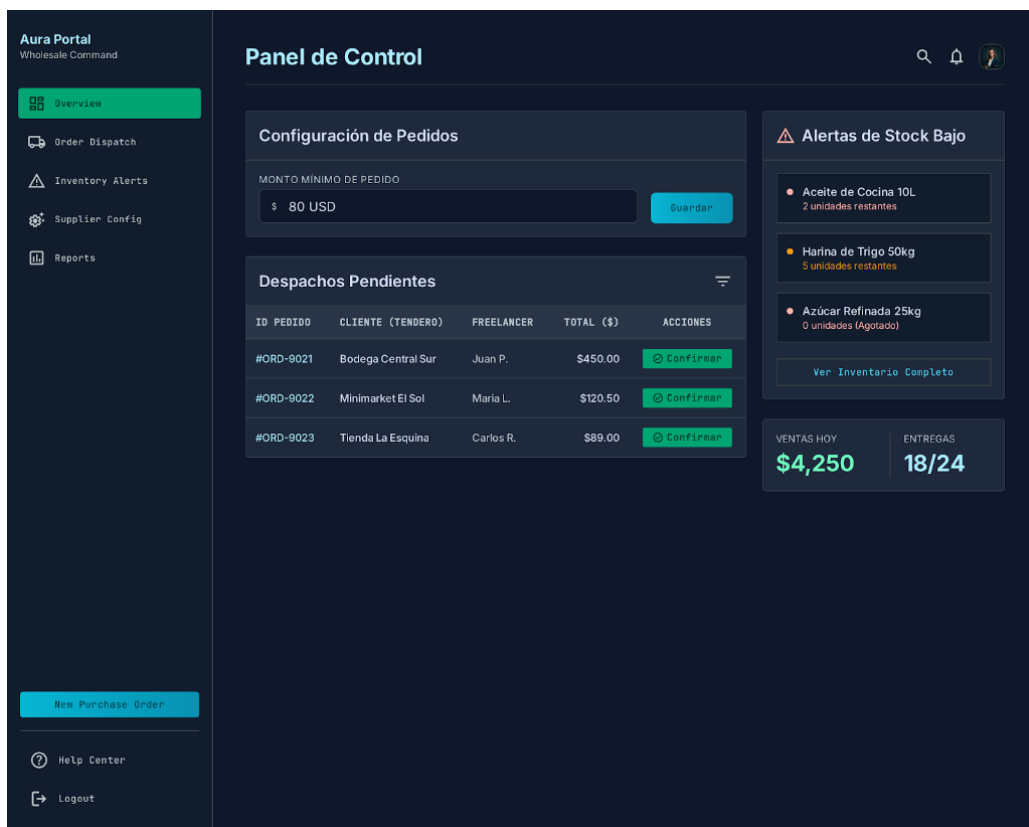


Figura 2.7: Prototipo 5: Panel de Control de Administración del Proveedor.

Bibliografía

- [1] Vercel. Next.js Documentation. 2026. <https://nextjs.org/docs>
- [2] Django Software Foundation. Django Documentation. 2026. <https://docs.djangoproject.com>
- [3] Encode Collaborative. Django REST Framework Documentation. 2026. <https://www.django-rest-framework.org>
- [4] The PostgreSQL Global Development Group. PostgreSQL 16 Documentation. 2026. <https://www.postgresql.org/docs/16/index.html>
- [5] Stripe. Stripe API Reference. 2026. <https://stripe.com/docs/api>
- [6] Pressman, Roger S. Software Engineering: A Practitioner's Approach. McGraw-Hill Education, 8th edition, 2014.
- [7] Schwaber, Ken and Beedle, Mike. Agile Software Development with Scrum. Prentice Hall, 2002.